FUNDAMENTOS DE TERRAFORM · CLASES 4 Y 5

Resumen: HCL, estructura de proyecto, LocalStack y meta-argumentos

Instructor: Ing. Álvaro Socop

1. ¿Qué es realmente un archivo .tf?

Los archivos `.tf` son la unidad básica de configuración de Terraform. Son **texto plano** (cualquier editor sirve), **declarativos** (describes el QUÉ, no el CÓMO), **versionables** en Git y se guardan en **UTF-8**.

> **Para recordar**
>
> Terraform lee **todos** los `.tf` de una carpeta y los junta como si fueran un solo archivo. Por eso puedes separar tu código en `providers.tf`, `variables.tf`, `main.tf`… y todo funciona igual.

Todo archivo `.tf` se compone de **bloques** con esta forma:

```
tipo "etiqueta1" "etiqueta2" {
  argumento = valor
}

resource "aws_instance" "web_server" {   # tipo = resource
  ami           = "ami-0c55b159cbfafe1f0" # etiqueta1 = aws_instance
  instance_type = "t2.micro"              # etiqueta2 = web_server
}
```

2. Convenciones y estructura de proyecto

Archivo	Qué contiene
`main.tf`	Resources principales: el punto de entrada.
`providers.tf`	Bloque `terraform { }` y configuración de los providers.
`variables.tf`	**Declaración** de las variables de entrada (tipo, descripción, default).
`locals.tf`	Valores calculados internos (`local.nombre`).
`outputs.tf`	Valores que se muestran al terminar el apply.
`terraform.tfvars`	Los **valores** de las variables. Se lee solo en cada plan/apply. **No va a Git.**
`backend.tf`	Dónde se guarda el state remoto (clases posteriores).

> **Regla de oro**
>
> **A Terraform no le importa el nombre del archivo, a tu equipo sí.** Usa siempre los nombres estándar para que cualquiera entienda tu proyecto en segundos.

Un `.gitignore` mínimo para cualquier proyecto Terraform (en `.gitignore` los comentarios van en su propia línea):

```
.terraform/
*.tfstate
*.tfstate.*
*.tfvars
!*.tfvars.example
```

El truco de `!*.tfvars.example`: versionas un archivo de **ejemplo** para que tu equipo sepa qué valores llenar, y cada quien crea su `terraform.tfvars` real con `Copy-Item terraform.tfvars.example terraform.tfvars`.

3. Sintaxis HCL: las reglas que más errores causan

- **Asignación con un solo `=`.** El doble `==` es para comparar.
- **Strings solo con comillas dobles `"..."`.** Las comillas simples son inválidas.
- **HCL distingue mayúsculas:** `resource` funciona, `Resource` no.
- Bloques entre llaves `{ }`, un argumento por línea, sin punto y coma.
- Comentarios con `#` (una línea) o `/* ... */` (varias líneas).
- Textos largos con heredoc: `<<-EOT ... EOT`.
- Indentación de 2 espacios: `terraform fmt` la pone por ti.

4. Tipos de datos

Tipo	Ejemplo	Cómo se usa
string	`"us-east-1"`	Texto.
number	`42`, `3.14`	Números enteros o decimales.
bool	`true` / `false`	Sí o no.
list	`["a", "b", "c"]`	Ordenada, por índice: `var.zonas[0]`.
map	`{ dev = "t3.micro" }`	Clave = valor: `var.tipos["dev"]`.
set	`[80, 443, 80]` → `[80, 443]`	Sin duplicados y **sin índice**: `var.puertos[0]` da error.
object	`{ nombre = "web", cpu = 2 }`	Como un map, pero cada atributo con su tipo: `var.servidor.nombre`.
tuple	`["backup", 7, true]`	Como una list, pero cada posición con su tipo.
null	`null`	Ausencia de valor.

> **Tu laboratorio: terraform console**
>
> Antes de escribir una expresión en un `.tf`, pruébala en `terraform console`: escribes `var.zonas[1]` y responde `"us-east-1b"`. Sales con `exit`.

5. Bloques y version constraints

Bloque	Para qué sirve
`terraform { }`	Configuración global: versión de Terraform y providers requeridos.
`provider "aws" { }`	Configura la conexión con el proveedor.

Bloque	Para qué sirve
`resource "tipo" "nombre" { }`	Crea y administra infraestructura.
`data "tipo" "nombre" { }`	Solo **lee** algo que ya existe (no crea nada).
`variable "nombre" { }`	Entrada parametrizable: `var.nombre`.
`output "nombre" { }`	Expone un valor al terminar el apply.
`locals { }`	Valores calculados internos: `local.nombre`.
`module "nombre" { }`	Reutiliza un conjunto de recursos (más adelante en el curso).

Constraint	Acepta	No acepta
`= 1.5.0`	Solo la 1.5.0	Todo lo demás
`>= 1.5.0`	1.5.0 en adelante (incluida la 2.0)	Menores a 1.5.0
`~> 1.5`	1.5, 1.6 … 1.99	2.0
`~> 1.5.0`	1.5.0, 1.5.1 … (solo parches)	1.6.0

Truco para recordarlo: `~>` **solo deja subir el último número que escribiste.**

6. terraform fmt y terraform validate

	terraform fmt	terraform validate
Revisa	El **estilo**: indentación, espacios, `=` alineados	La **lógica**: argumentos, referencias, tipos
¿Modifica archivos?	Sí (con `-check` no)	Nunca
¿Necesita init?	No	Para errores de sintaxis no; para lógica sí
Opciones útiles	`-check` (para CI/CD), `-diff`, `-recursive`	—

> **Los errores salen por rondas**
>
> Terraform no siempre muestra todos los errores de una vez. Corrige, vuelve a correr `terraform validate`, y repite hasta ver `Success! The configuration is valid.`

7. Ejercicio: encuentra los 5 errores

```
Resource "aws_instance" "WebServer" {
  ami = ami-0c55b159cbfafe1f0
  instance_type == "t2.micro"
  tags = {
    Name = 'Mi Servidor'
  }
}
```

Error	Corrección	¿Quién lo detecta?
`Resource`	`resource`	`validate`: «Did you mean "resource"?»
`instance_type ==`	`instance_type =`	`validate`: «Argument or block definition required»

Error	Corrección	¿Quién lo detecta?
`'Mi Servidor'`	`"Mi Servidor"`	`validate`: «Single quotes are not valid»
`ami = ami-0c55...`	`ami = "ami-0c55..."`	`validate` después de `init`: «Invalid reference»
`"WebServer"`	`"web_server"`	**Nadie.** Es convención (snake_case): Terraform dice `Success!`

Lección: que Terraform diga `Success!` no significa que el código siga las buenas prácticas. Las convenciones las cuida el equipo.

8. LocalStack: AWS en tu computadora

Trabajar siempre contra la nube real tiene problemas: **costos** por recursos olvidados, **esperas** de minutos, **dependencia de internet** y compañeros **chocando** en la misma cuenta. **LocalStack** emula la API de AWS (S3, EC2, DynamoDB, Lambda, SQS…) dentro de un contenedor de Docker, en el puerto **4566**.

```
# En la carpeta donde está docker-compose.yml
$env:LOCALSTACK_VERSION = "4.14.0"
docker compose up -d
curl.exe http://localhost:4566/_localstack/health   # "s3": "available"...
docker compose down                                 # apagar (se borra todo)
```

> **Importante (2026)**
>
> Desde la versión 2026.03, la imagen más reciente de LocalStack pide una cuenta gratis (plan Hobby) y un token en `LOCALSTACK_AUTH_TOKEN`. En el curso usamos la versión **4.14.0**, que no lo pide.

Para usarlo con Terraform, el código de los resources **no cambia**. Solo cambia el provider:

```
provider "aws" {
  region     = "us-east-1"
  access_key = "test"     # credenciales falsas: solo para LocalStack
  secret_key = "test"

  skip_credentials_validation = true
  skip_metadata_api_check     = true
  skip_requesting_account_id  = true
  s3_use_path_style           = true

  endpoints {
    s3  = "http://localhost:4566"
    ec2 = "http://localhost:4566"
    sts = "http://localhost:4566"
  }
}
```

Para ver tus recursos con la AWS CLI agrega `--endpoint-url=http://localhost:4566`, por ejemplo `aws --endpoint-url=http://localhost:4566 s3 ls`. La herramienta `tflocal` (`pip install terraform-local`) configura los endpoints automáticamente.

9. Meta-argumentos

Son argumentos especiales que funcionan en **cualquier** resource, sin importar el provider.

Meta-argumento	Qué hace	Dirección resultante
`count`	Crea N copias. `count.index` va de 0 a N-1.	`aws_s3_bucket.b[0]`, `[1]`…
`for_each`	Crea una copia por cada clave de un map o set. Usa `each.key` y `each.value`.	`aws_s3_bucket.b["testing"]`
`depends_on`	Fuerza un orden. Solo cuando **no** hay una referencia que cree la dependencia.	—
`lifecycle`	Controla cómo se crea, cambia o destruye.	—
`provider`	Elige otro provider o región para ese resource.	—

Recurso condicional: `count = var.crear ? 1 : 0` hace que el recurso exista o no según una variable.

```
resource "aws_s3_bucket" "buckets_con_count" {
  count  = var.cantidad_buckets                  # 3 copias
  bucket = "mi-app-count-${count.index + 1}"     # -1, -2, -3
}

resource "aws_s3_bucket" "buckets_con_foreach" {
  for_each = var.buckets_por_ambiente           # map: desarrollo, testing...
  bucket   = "${each.value}-${var.ambiente}"
}
```

10. La trampa de count con listas

Con `equipos = ["frontend", "backend", "datos"]` creamos un bucket por equipo de dos formas. Luego quitamos solo `"frontend"`:

	Con count	Con for_each
Cómo identifica	Por **posición**: [0], [1], [2]	Por **clave**: ["frontend"], ["backend"]…
Al quitar frontend	Los índices se corren: backend y datos se **destruyen y recrean**, y [2] se borra	Solo se destruye `["frontend"]`
Plan	2 recreados + 1 destruido	1 destruido

> **Regla práctica**
>
> Si cada elemento tiene identidad propia (un equipo, un ambiente, un usuario), usa **for_each**. Usa **count** para copias idénticas o para el truco condicional `? 1 : 0`. Para usar una lista con `for_each`, conviértela con `toset(var.lista)`.

11. lifecycle

Opción	Qué hace
`prevent_destroy = true`	Terraform se **niega** a destruir el recurso (`Instance cannot be destroyed`). Ideal para bases de datos de producción. Solo acepta `true`/`false` literal, no variables.
`ignore_changes = [tags["Descripcion"]]`	Si alguien cambia ese atributo fuera de Terraform, no se marca como drift ni se corrige.
`create_before_destroy = true`	En un reemplazo, crea el nuevo **antes** de borrar el viejo (evita caídas).

12. Expresiones e interpolación

Expresión	Resultado
`"web-${var.ambiente}"`	`"web-dev"`
`var.ambiente == "prod" ? "t3.large" : "t3.micro"`	`"t3.micro"`
`lookup(var.tipos, "qa", "t3.micro")`	`"t3.small"` (o el default si no existe)
`merge({a = 1}, {b = 2})`	`{ a = 1, b = 2 }`
`join("-", ["a", "b"])`	`"a-b"`
`split("-", "us-east-1")`	`["us", "east", "1"]`
`length(var.zonas)`	`3`
`[for z in var.zonas : upper(z)]`	`["US-EAST-1A", ...]`

Cómo leer recursos con count y for_each en outputs:

- `aws_s3_bucket.b[*].bucket` (**splat**): lista con el atributo de todas las copias de un `count`.
- `{ for k, b in aws_s3_bucket.b : k => b.bucket }`: convierte un `for_each` en un map clave → nombre.
- `try(aws_s3_bucket.b[0].bucket, "No creado")`: evita el error si un recurso condicional no existe.

13. Buenas prácticas

- **snake_case** en todo: recursos, variables y outputs (`web_server`, no `WebServer`).
- **Nombres descriptivos:** `bloqueo_publico`, no `bpa1`. Si solo hay uno de su tipo, la comunidad usa `this`.
- `terraform fmt` **antes de cada commit** (o «Format on Save» en VS Code).
- `locals` para no repetir la misma expresión en varios lugares.
- `validation` en variables para frenar valores inválidos antes de crear nada.
- **Nunca secretos en un** `.tf` ni en Git. Las credenciales `test/test` solo son válidas porque son de LocalStack.

14. Ejercicio estrella

- Corre el lab de la Clase 3 (VPC, subnet, Internet Gateway, route table, security group, EC2 y bucket S3) en **LocalStack**. Solo cambia `providers.tf`.
- Agrega `count` a la instancia con una variable `cantidad_servidores` y muestra los IDs con `[*]`.
- Provoca un drift sin consola web y detéctalo con `terraform plan`:

```
$id = (terraform output -json instance_ids | ConvertFrom-Json)[0]
aws --endpoint-url=http://localhost:4566 ec2 create-tags `
  --resources $id --tags Key=Name,Value=hacked-server
terraform plan     # ~ "Name" = "hacked-server" -> "edutek-web-1"
```

- Al terminar: `terraform destroy` y `docker compose down`.

15. Comandos nuevos

Comando	Para qué sirve
`terraform fmt -check`	Revisa el formato sin modificar (falla si hay algo mal: ideal en CI/CD).
`terraform fmt -diff`	Formatea y muestra qué cambió.
`terraform validate`	Revisa sintaxis y lógica sin contactar a la nube.
`terraform console`	Prueba expresiones, funciones y variables en vivo.
`terraform plan -var="x=valor"`	Cambia el valor de una variable solo para ese comando.
`docker compose up -d` / `down`	Enciende / apaga LocalStack.

> ***"Si solo aprenden sintaxis de Terraform, el curso falla. Si aprenden a pensar infraestructura como código, pueden automatizar cualquier nube."***